

CS5617 Software Engineering: Sample quiz

Total: 16 marks

Time: 55 minutes

Question #1: (1 mark)

Name the SDLC phase where the system is deployed to the production environment. Give one key activity in that phase.

Question #2: (1 mark)

In C# or Java, what is the difference between an **interface** and an **abstract class**? Give one example scenario where you would prefer each.

Question #3: (1 mark)

What is Open-Closed principle in OOD. Do final classes (in Java) / Sealed classes (in C#) violate this principle?

Question #4: (1 mark)

You are designing a **Notification System** that can send alerts via Email, SMS, or Push Notification. Which creational pattern would you use to instantiate the appropriate notifier at runtime?

Question #5: (2 marks)

Draw a class diagram to show the relationship between the various types here. The type names are self-explanatory.

- interface IAutomobile,
- class Engine,
- class Chassis,
- class SnowTyre (to be used only during snow),
- class Passenger
- class Car.

Question #6: (2 marks)

A **Help Desk Ticket System** routes incoming tickets through multiple handlers: Level1Support, Level2Support, and Manager. Each handler decides whether to process the ticket or pass it along.

1. Which **behavioral design pattern** is this? *(0.5 mark)*
2. Draw an **activity diagram** showing the flow of ticket handling. *(1.5 marks)*

Question #7: (3 marks)

In a software project similar to yours, a whiteboard module is offering an interface with a “Draw” method to render the whole whiteboard onto the canvas where as internally the component might “Draw” one or more shapes (like Circles, Rectangles, Squares etc.) onto the canvas. The client of the whiteboard component only cares about calling the “Draw” method on the whiteboard object itself. Which design pattern is used here? Draw the class diagram for the types mentioned here.

Question #8: (5 marks)

Design a test harness that will support running automated tests for a software project.

Hint: You can think of three major components for a test harness:

1. An interface that defines how a test class should look like.
2. Various test classes each implementing the interface mentioned above.
3. Execution engine that will run all the selected tests.

You can see that your framework will define and implement #1 and #3 above where as consumers of your framework will be writing #2.

Questions:

- Write the interface definition for #1. *(1 out of 5 marks)*
- Write skeletal code for #3. *(2 out of 5 marks)*
- Draw the class diagram for the test harness. *(2 out of 5 marks)*

Answer keys for sample quiz

Q1 – SDLC Phase & Key Activity

- *Answer:* **Deployment phase.**
- *Key activity:* Installing the system in the production environment, configuring it, and performing final acceptance testing.

Q2 – Interface vs Abstract Class (C#/Java)

- **Interface:** Defines a contract (methods/properties) without implementation. A class can implement multiple interfaces. *Example use:* When unrelated classes must share the same capability (e.g., IComparable for sorting).
- **Abstract Class:** Can have both abstract methods and concrete methods; supports shared state and partial implementation. A class can only inherit from one abstract class. *Example use:* When related classes share common code but also need to enforce certain abstract methods (e.g., Shape base class with Draw()).

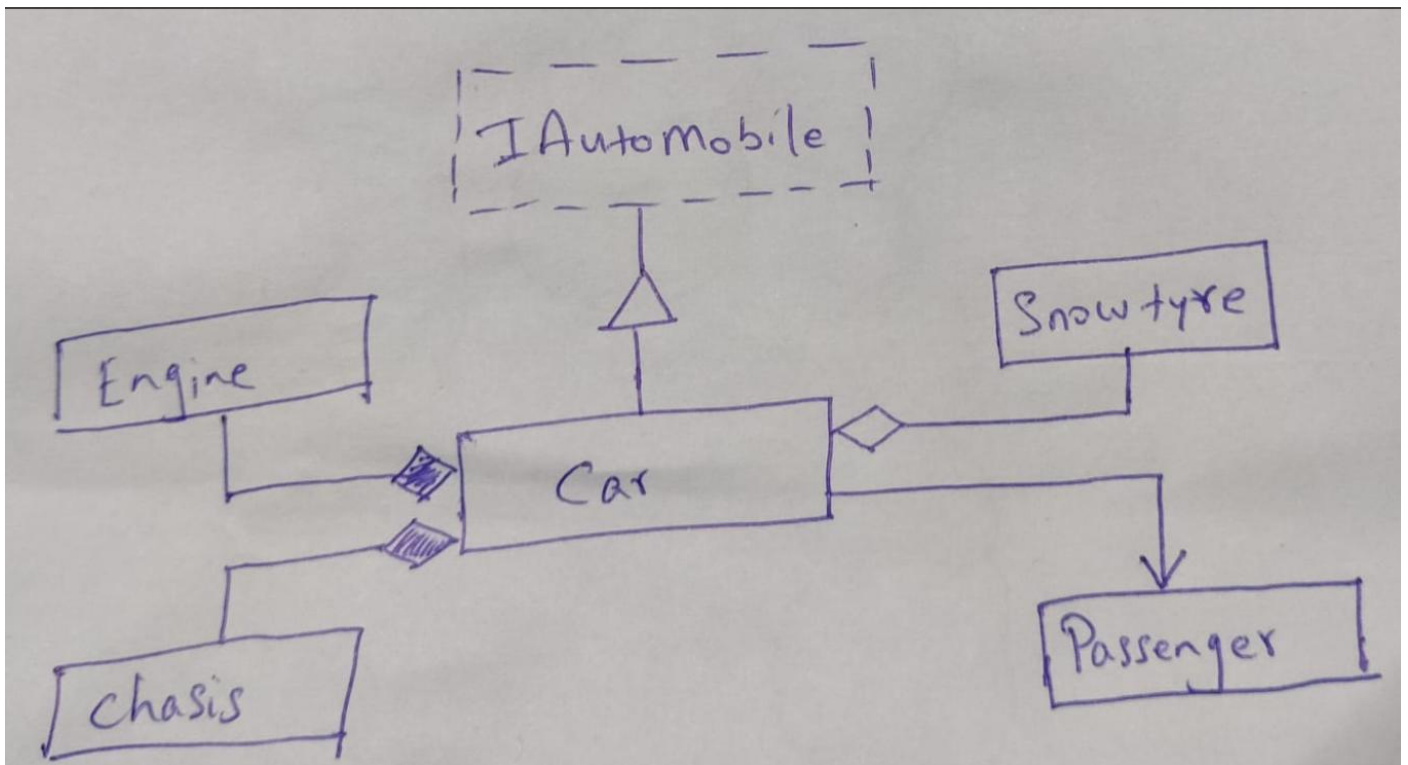
Q3 – Open-Closed Principle & Final/Sealed Classes

- **Open-Closed Principle (OCP):** Software entities should be **open for extension** but **closed for modification**.
- **Final (Java) / Sealed (C#) classes:** They restrict inheritance, so they can limit extension via subclassing. However, OCP can still be satisfied if extension is achieved through composition, interfaces, or other patterns.

Q4 – Creational Pattern for Notification System

- *Answer:* **Factory Method** or **Abstract Factory** pattern.
- *Reason:* Allows creation of the appropriate notifier (Email/SMS/Push) at runtime without changing client code.

Q5 – Class Diagram

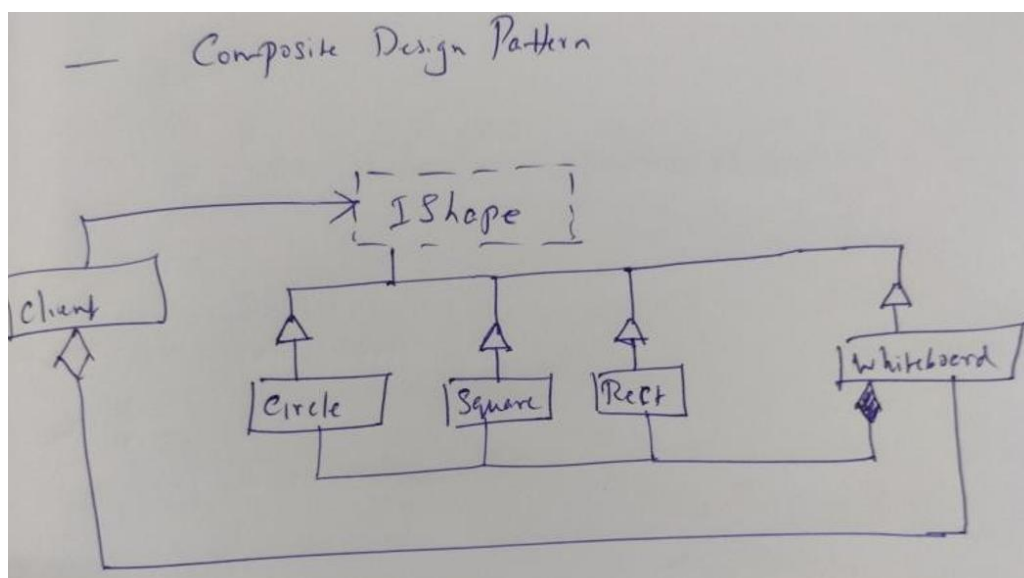


Q6 – Help Desk Ticket System

- Pattern: Chain of Responsibility.
- Activity Diagram (hint only):

Start → Ticket received → Level1Support: process or pass to Level2Support → Level2Support: process or pass to Manager → Manager processes → End.

Q7 – Whiteboard “Draw” Method



Q # 1

#1 interface ITest

{

bool Run()

int priority { get; }

// other parameters for a test like category

}

#2 class TestHarness

{

Void Execute()

{

ITest test1 = new Test1();

ITest test2 = new Test2();

//

List<ITest> tests;

test.Add(test1);

test.Add(test2);

// Better yet, read from a config file.

foreach (ITest test in tests)

{
result = test.Run();

}

// Book keep various test results

}

